

ML-RT2: advanced ML methods for emulating cosmological radiative transfer

A methods brief for the team

July 29, 2026

Purpose: a shareable, non-code overview of the methods we are trialing for the third paper. Each method gets a plain-language description, the key equation with its symbols spelled out, and why it is interesting for us. All eight models are implemented as prototypes in a shared training/evaluation framework (dataset 053), ready for the GPU cluster.

How to read the diagrams. The colours are consistent throughout: *pink* = data / vectors (inputs, outputs, intermediate representations); *blue* = learned networks / operators; *orange* = special operations or physics; *green* = loss (training) terms.

1 Context and goal

- Prior work (papers 1–2) showed that feed-forward and generative networks (MLP, CVAE, CGAN, LSTM) can emulate STARDUST radiative-transfer profiles (Krause et al., 2018); emulating expensive cosmological simulations with neural networks is now common practice (Kern et al., 2017).
- Paper 3 targets **methods not yet widely translated into astrophysical radiative transfer**, kept **comparable** to papers 1–2 (same data, split and metrics) and studied for **how they optimise** (training behaviour, data efficiency, cost).
- We want emulators that are *capable* — accurate, and ideally carrying an uncertainty estimate and/or respecting the physics — but *cheap to evaluate* once trained.

2 Problem setup

- STARDUST is a deterministic function that turns $P = 8$ physical parameters θ (log halo mass, redshift, source age, QSO spectral slope and efficiency, stellar escape fraction, IMF slope and minimum mass) into four radial profiles $\mathbf{u}(r) = (x_{\text{HII}}, x_{\text{HeII}}, x_{\text{HeIII}}, T)$ — the ionised fractions of hydrogen and helium and the gas temperature, each sampled at $L = 1500$ radii r .
- The emulator learns an approximation $\hat{\mathbf{u}} = f_{\phi}(\theta)$ (with ϕ the network weights) from dataset 053 ($N = 41,081$ examples, split 80/10/10 into train/val/test).
- Pre-processing matches papers 1–2: parameters rescaled to $[0, 1]$; ionised fractions passed through $\log_{10}(x + 10^{-6})$; temperature through $\log_{10} T$; then each channel is standardised (zero mean, unit variance).

- Key difficulty: the profiles are smooth except for a **sharp ionisation front** (where the gas flips from ionised to neutral). Most networks have a low-frequency bias and tend to *smooth over* exactly this front, so representing it well is a recurring theme below.

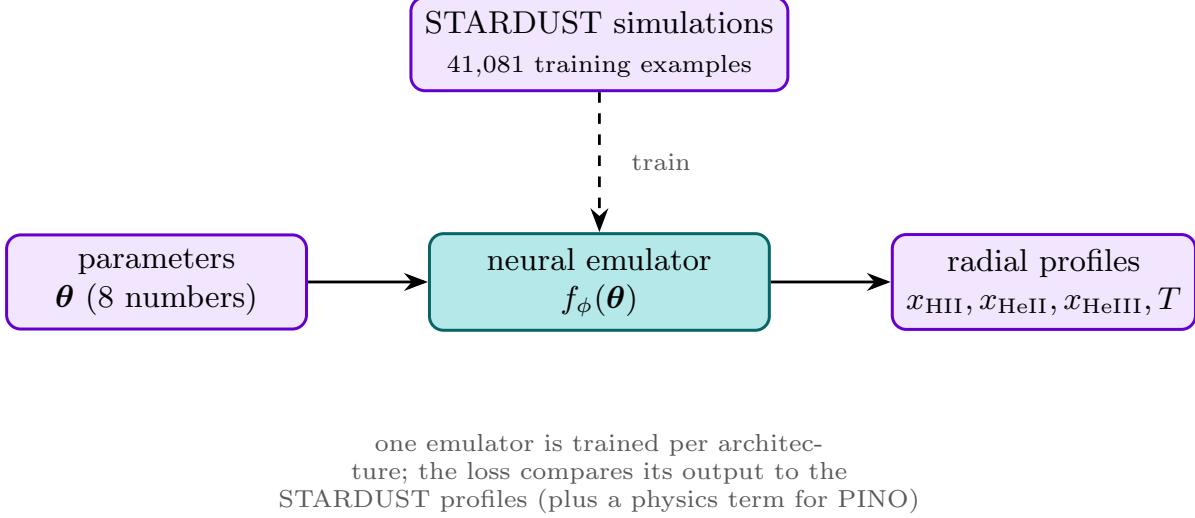


Figure 1: The shared emulation pipeline. Every architecture maps the 8 physical parameters to the four radial profiles; all are trained on the same STARDUST dataset (053) and compared with the same metrics, so any difference reflects the *method*, not the setup.

3 Methods

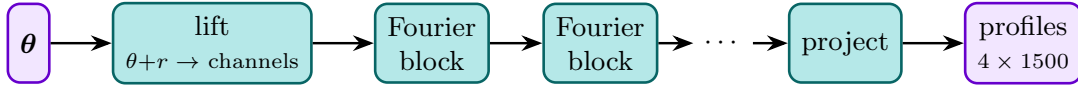
Three of the models below (FNO, DeepONet, PINO) are *neural operators*. An operator is a map from a function to a function; here the thing we want to predict *is* a function — the profile as a function of radius, $\mathbf{u}(r)$ — so this framing is natural (Kovachki et al., 2023). Since our input is only 8 numbers, not a function, we first “**lift the parameter vector to a field**”: we build a table with one row per radius point and, in every row, write the same 8 parameters plus that point’s radius r . The input is then a function over radius that an operator can transform — intuitively, we *paint* the parameters across the radial grid and let the network reshape that painted field into the output profiles. Each architecture gets its own page below.

3.1 Fourier Neural Operator (FNO)

How it works. The FNO edits the field in “wavelength space” (Li et al., 2021): each layer transforms the field, keeps and reweights only its smooth large-scale waves, transforms back, and adds a local correction,

$$v_{\ell+1} = \sigma(W_\ell v_\ell + \mathcal{F}^{-1}(R_\ell \mathcal{F} v_\ell)). \quad (1)$$

- v_ℓ is the field (a stack of feature channels over radius) at layer ℓ ; $\mathcal{F} / \mathcal{F}^{-1}$ are the Fourier transform and its inverse, which decompose the field into waves of different wavelengths and back again; R_ℓ is a set of learned weights applied only to the lowest K wave-modes (the large, smooth scales); W_ℓ is a simple per-radius linear mix; and σ is a nonlinearity.
- **Why for us:** working with wavelengths captures the big smooth shape of the profile very efficiently, while the local term helps with the sharp front. It is resolution-free (train at 1500 points, evaluate at any grid) and extremely cheap at inference — a strong “capable and fast” baseline barely used in astrophysical RT.



one Fourier block: $v_{\ell+1} = \sigma(W v_\ell + \mathcal{F}^{-1}(R_\ell \mathcal{F} v_\ell))$

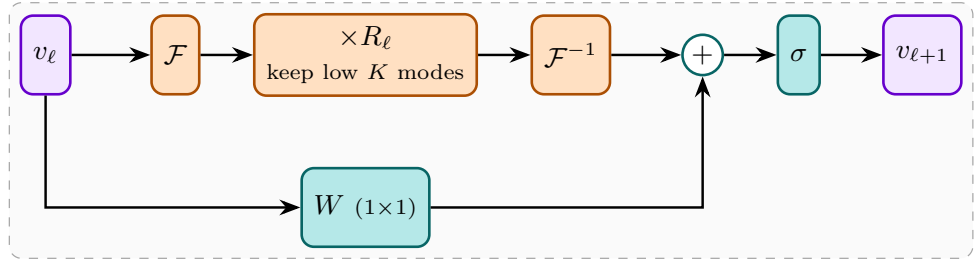


Figure 2: **Fourier Neural Operator.** The 8 parameters are lifted to a field over radius, then passed through a few Fourier blocks. Each block (expanded below) has a spectral path that keeps the smooth large-scale structure ($\mathcal{F} \rightarrow \times R_\ell \rightarrow \mathcal{F}^{-1}$) plus a local per-radius term (W); a final projection gives the four profiles.

3.2 DeepONet

How it works. DeepONet builds the profile from learned building-block curves (Lu et al., 2021):

$$\hat{u}_c(r) = \sum_{k=1}^K b_{c,k}(\boldsymbol{\theta}) t_k(r) + \beta_c. \quad (2)$$

- The *trunk* network turns a radius r into K basis shapes $t_k(r)$ (a shared library of building-block curves); the *branch* network turns the parameters into coefficients $b_{c,k}(\boldsymbol{\theta})$ that say how much of each building block to use for output channel c ; β_c is an offset. The profile is their weighted sum.
- **Why for us:** because the output is written as a function of r , we can query the profile at *any* radius, and evaluation is only a few small matrix multiplies (very fast). We feed the trunk Fourier features of r (Tancik et al., 2020) so it can represent the sharp front.

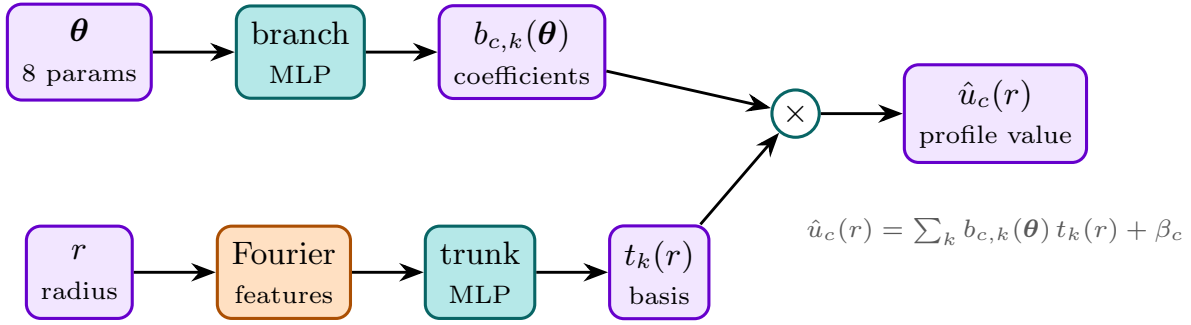


Figure 3: **DeepONet.** The branch network turns the parameters into coefficients and the trunk network turns a radius into basis shapes; their weighted sum is the profile value at that radius, so the profile can be queried at any radius.

3.3 Physics-informed neural operator (PINO)

How it works. Train an operator on the data *and* nudge it to obey the radiative-transfer physics (Raissi et al., 2019; Li et al., 2024). The loss has two parts,

$$\mathcal{L} = \underbrace{\|\hat{\mathbf{u}} - \mathbf{u}\|^2}_{\text{data: match STARDUST}} + \lambda \underbrace{\|\mathcal{R}[\hat{\mathbf{u}}]\|^2}_{\text{physics: obey RT law}}, \quad (3)$$

where $\mathcal{R}[\hat{\mathbf{u}}]$ measures how much the prediction *violates* a physical law and λ sets how strongly we enforce it. The law we use is ionisation equilibrium: the rate at which photons ionise atoms balances the rate at which electrons recombine,

$$\underbrace{\Gamma_0(\boldsymbol{\theta}) \frac{e^{-\tau(r)}}{r^2} (1 - x_{\text{HII}})}_{\text{photo-ionisation}} = \underbrace{\alpha_{\text{HII}}(T) x_{\text{HII}}^2}_{\text{recombination}}, \quad \tau(r) = \kappa(\boldsymbol{\theta}) \int_0^r (1 - x_{\text{HII}}) dr'. \quad (4)$$

- x_{HII} is the ionised-hydrogen fraction and $1 - x_{\text{HII}}$ the neutral fraction; $\tau(r)$ is the optical depth (how much ionising light has been absorbed on the way out to r); $\alpha_{\text{HII}}(T)$ is the temperature-dependent recombination coefficient (from Krause et al. 2018, following Fukugita & Kawasaki); and Γ_0, κ are two small per-sample numbers predicted from $\boldsymbol{\theta}$ that absorb the unknown source brightness and gas density — so it is the *shape* of the law, not its scale, that regularises. We add mild “sanity” terms too (fractions in $[0, 1]$, helium conservation, the front not turning back on).
- **Why for us:** anchoring the physics to data is far more robust than a pure physics-only network (our earlier pure-PINN attempt never converged, as expected for this stiff, front-forming system). Here the operator already fits the data and the physics only refines it, most helpfully at the front where data is thin. Setting $\lambda = 0$ recovers the plain operator, a built-in ablation; helium/heating terms are clean extensions.

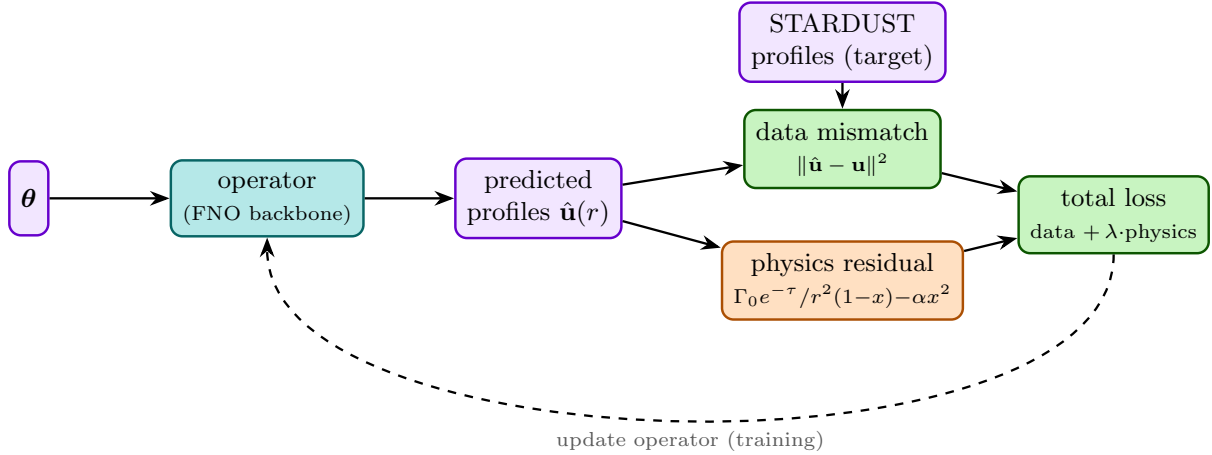


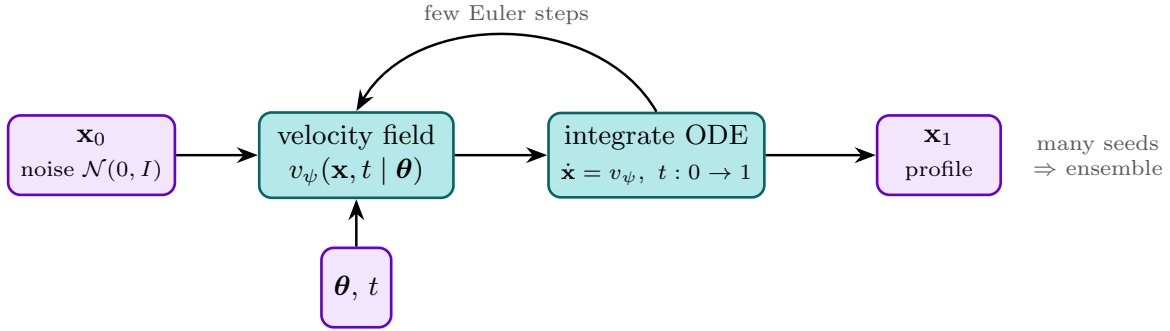
Figure 4: **PINO**. The operator predicts the profiles (right). Training uses two terms of one loss: a *data* term comparing the prediction to the STARDUST target, and a *physics* term measuring how far the prediction is from the ionisation-equilibrium law. Their sum updates the operator; with $\lambda = 0$ only the data term remains (a plain operator).

3.4 Conditional flow matching (generative, with uncertainty)

How it works. Learn a “current” that carries random noise into valid profiles. A velocity field $v_\psi(\mathbf{x}, t \mid \boldsymbol{\theta})$ says, for a partly-formed profile $\mathbf{x}$ at pseudo-time $t \in [0, 1]$ (given the parameters), which direction to nudge it. We define a straight path from noise $\mathbf{x}_0 \sim \mathcal{N}(0, I)$ to a real profile $\mathbf{x}_1$, namely $\mathbf{x}_t = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1$, and train the field to point along it (Lipman et al., 2023; Liu et al., 2023):

$$\mathcal{L} = \mathbb{E}_{t, \mathbf{x}_0, \mathbf{x}_1} \left\| v_\psi(\mathbf{x}_t, t \mid \boldsymbol{\theta}) - (\mathbf{x}_1 - \mathbf{x}_0) \right\|^2. \quad (5)$$

- The target direction is simply “from noise towards data”, $\mathbf{x}_1 - \mathbf{x}_0$. To generate a profile we start from noise and follow the current in a few small steps.
- **Why for us:** drawing several different noise starts gives several plausible profiles — an *ensemble*, hence an error bar. It is the generative successor to the paper-2 CVAE/CGAN but adversary-free (more stable), and few-step sampling keeps inference cheap. (Diffusion models, Ho et al. 2020, are the closely related but more expensive cousin.)



training: fit v_ψ to the straight-line velocity $\mathbf{x}_1 - \mathbf{x}_0$ along $\mathbf{x}_t = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1$

Figure 5: **Conditional flow matching.** A network (the velocity field), conditioned on the parameters and pseudo-time, is applied repeatedly to carry Gaussian noise to a profile. Different random starts give an ensemble, hence calibrated uncertainty.

3.5 Profile transformer (secondary)

How it works. We chop the profile into radial *patches* and introduce a learnable “query” token q_i for each patch, plus one token carrying the parameters. A transformer lets all tokens exchange information through self-attention (Vaswani et al., 2017; Cao, 2021), after which each query decodes its own patch; the patches are stitched into the full profiles.

- **Why for us:** attention naturally couples long-range structure — e.g. the position of the front depends on the total absorption interior to it — and lets the four species inform one another. A fresh architecture for this problem.

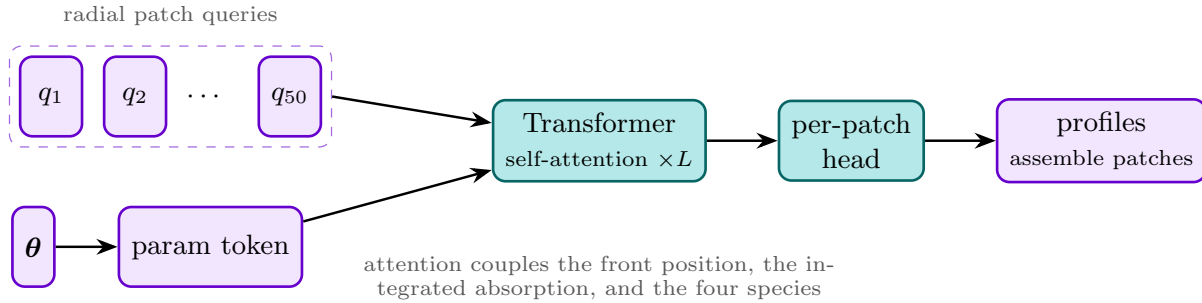


Figure 6: **Profile transformer.** A parameter token and one learnable query per radial patch attend to one another; each query then outputs its patch of the profile, and the patches are assembled into the full curves.

3.6 Neural-ODE decoder in radius

How it works. Build the profile by “marching outward” (Chen et al., 2018). A latent state $\mathbf{h}$ is initialised from the parameters and updated step by step in radius by a learned rate f_ϕ , with a readout g_ϕ giving the profile at each radius:

$$\frac{d\mathbf{h}}{dr} = f_\phi(\mathbf{h}, r, \boldsymbol{\theta}), \quad \mathbf{u}(r) = g_\phi(\mathbf{h}(r)). \quad (6)$$

- **Why for us:** this mirrors how the real ionisation structure builds up with distance from the source — a physically-structured, “marching” emulator.

march outward: $\mathbf{h} \leftarrow \mathbf{h} + dr f_\phi(\mathbf{h}, r, \boldsymbol{\theta}), \quad \mathbf{u}(r) = \text{readout}(\mathbf{h})$

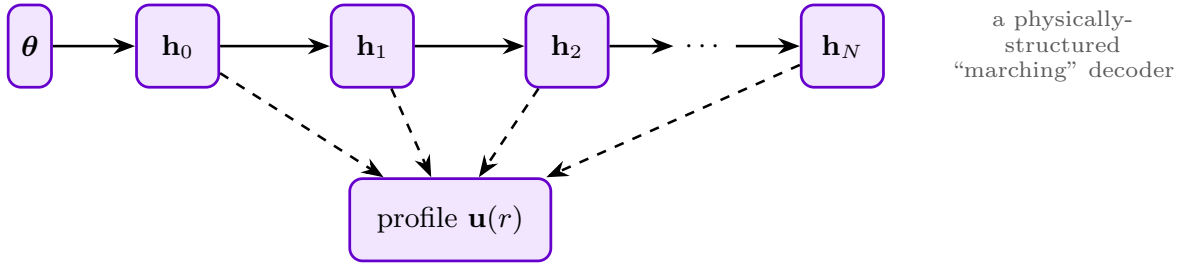


Figure 7: **Neural-ODE decoder.** A latent state initialised from the parameters is integrated outward in radius by a learned update; a readout at each step gives the profile.

3.7 Joint-embedding predictive architecture (JEPA)

How it works. Learn to predict an *abstract summary* of the profile before decoding it (Assran et al., 2023). An encoder E_{θ} turns the parameters into an embedding, a predictor P maps that to a predicted profile-embedding $\hat{z}$, and a decoder turns $\hat{z}$ into a profile; a second encoder $E_{\mathbf{u}}$ provides the target embedding of the true profile. Training combines a latent-prediction loss and a reconstruction loss:

$$\mathcal{L} = \|\text{dec}(\hat{z}) - \mathbf{u}\|^2 + \beta \|\hat{z} - \text{sg}[E_{\mathbf{u}}(\mathbf{u})]\|^2, \quad \hat{z} = P(E_{\theta}(\theta)). \quad (7)$$

- $\text{sg}[\cdot]$ is a “stop-gradient” (we treat the target embedding as fixed, so the representation cannot collapse to something trivial) and β balances the two terms.
- **Why for us:** essentially unused in astronomy, and it yields a shared latent space useful for interpolation and data efficiency.

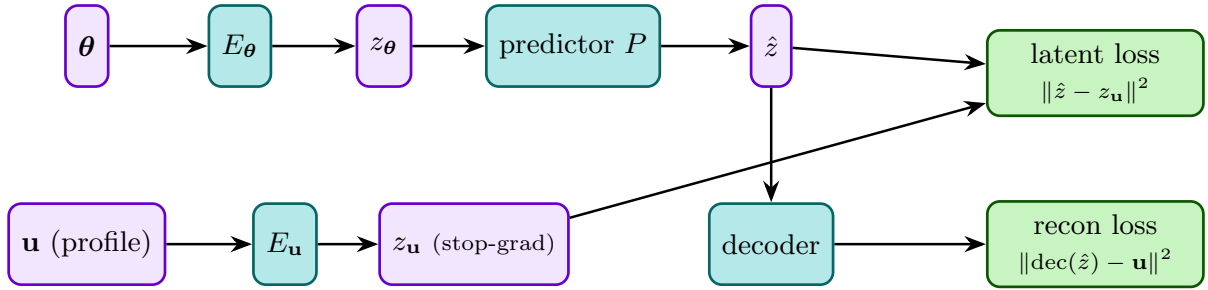


Figure 8: **JEPA**. The parameter path (top) predicts an embedding $\hat{z}$; the profile path (bottom) provides the target embedding $z_{\mathbf{u}}$. Training matches the two embeddings (latent loss) and reconstructs the profile from $\hat{z}$ (recon loss).

3.8 Conditional Neural Process (CNP)

How it works. A cheap route to a per-radius error bar (Garnelo et al., 2018). The parameters are turned into a representation, and a decoder reads (representation, radius) and outputs a Gaussian — a mean $\mu(r)$ and a spread $\sigma(r)$ — for the profile at that radius, trained by Gaussian likelihood:

$$\mathcal{L} = \frac{1}{2} \left\langle \left((\mathbf{u} - \mu(r)) / \sigma(r) \right)^2 + 2 \log \sigma(r) \right\rangle. \quad (8)$$

- This rewards accurate means *and* honest spreads, so $\sigma(r)$ grows where the model is genuinely uncertain (typically at the front).
- **Why for us:** a lightweight, distinct route to uncertainty (complementary to flow matching) that gives an error bar at every radius for almost no extra cost.

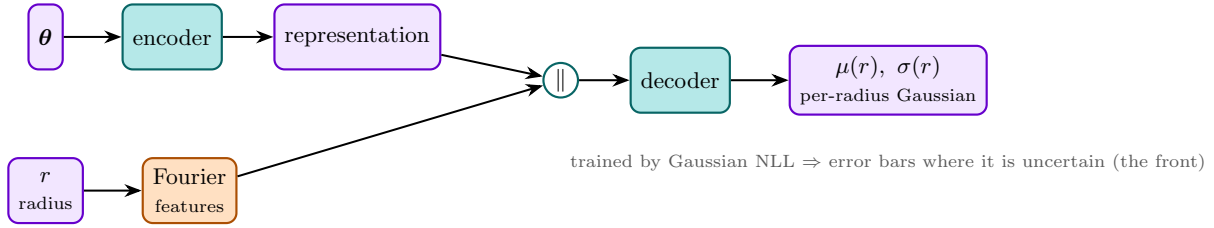


Figure 9: **Conditional Neural Process.** A parameter representation and the radius are decoded into a per-radius Gaussian (mean and spread), trained by Gaussian likelihood for calibrated uncertainty.

4 Evaluation and the optimisation study

- **Metrics:** log-space MSE and per-channel error (directly comparable to papers 1–2), a physical relative- L_2 error, an **ionisation-front-position error** (a sharp-feature metric that plain MSE can hide), and — for the generative/probabilistic models — uncertainty calibration (coverage, CRPS).
- **Fair protocol:** identical data split, compute budget and random seeds across all architectures, reporting mean $\pm$ std over 3 seeds, so comparisons are like-for-like.
- **Tuning strategy:** asynchronous successive halving (Li et al., 2020) via Optuna (Akiba et al., 2019) — many cheap exploratory trials on the fast consumer GPUs, full-budget refinement of the best configurations on the larger cards. Alongside this we run controlled one-parameter sweeps (learning rate, FNO modes, Fourier-feature bandwidth, physics weight λ) that *are* the “how do the models optimise” figures.

References

- Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *ACM SIGKDD*, 2019.
- Mahmoud Assran, Quentin Duval, Ishan Misra, et al. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- Shuhao Cao. Choose a transformer: Fourier or galerkin. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- Marta Garnelo, Dan Rosenbaum, Chris J. Maddison, et al. Conditional neural processes. In *International Conference on Machine Learning (ICML)*, 2018.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- N. S. Kern, A. Liu, A. R. Parsons, A. Mesinger, and B. Greig. Emulating simulations of cosmic dawn for 21 cm power spectrum constraints on cosmology, reionization, and x-ray heating. *Astrophysical Journal*, 848:23, 2017.
- Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with applications to pdes. *Journal of Machine Learning Research*, 24:1–97, 2023.
- F. Krause, R. M. Thomas, S. Zaroubi, and F. B. Abdalla. The effect of the ionizing background on cosmological reionization simulations (stardust). *New Astronomy*, 64:9, 2018.
- Liam Li, Kevin Jamieson, Afshin Rostamizadeh, et al. A system for massively parallel hyperparameter tuning. In *Proceedings of Machine Learning and Systems (MLSys)*, 2020.

- Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.08895.
- Zongyi Li, Hongkai Zheng, Nikola Kovachki, et al. Physics-informed neural operator for learning partial differential equations. *ACM/JMS Journal of Data Science*, 2024. arXiv:2111.03794.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.02747.
- Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2209.03003.
- Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3:218–229, 2021.
- M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear pdes. *Journal of Computational Physics*, 378:686–707, 2019.
- Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, et al. Fourier features let networks learn high frequency functions in low dimensional domains. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.